

Comparing CSP Replanning, MDP Value Iteration, and Q-Learning for Stochastic Grid Navigation

Milena Paques¹, Wael Drif¹, Rafael Palma Santos¹ Komildzhon Samadov¹

¹Université Libre de Bruxelles

milena.paques@ulb.be, wael.drif@ulb.be, rafael.palma.santos.da.silva.mendes@ulb.be, komildzhon.samadov@ulb.be

Abstract

This project compares three AI approaches on the same stochastic path planning task: a constraint-based replanning method, a model-based Markov Decision Process (MDP) solver, and model-free Reinforcement Learning (RL). The shared environment is a grid world with static obstacles, hazardous cells, and stochastic motion. We evaluate success rate, return, and runtime across difficulty levels and grid sizes. Results show that MDP is the most robust under high uncertainty, CSP replanning is strong on easier instances but degrades sharply at higher difficulty, and tabular Q-learning performs well when uncertainty is moderate but can fail on hard sparse-reward cases.

Code — <https://github.com/your-user/infoh410project>

1 Problem Formulation

The problem is a stochastic grid navigation task defined as follows:

- **Environment Map.** The environment is an $N \times N$ grid where cells are identified by their row and column coordinates (r, c) for $r, c \in \{0, \dots, N-1\}$. The grid contains a set of static obstacles O (impassable cells) and a set of hazards H (passable cells that incur an additional penalty).
- **Task.** An agent must navigate from a fixed start cell $s_0 = (0, 0)$ to a fixed goal cell $g = (N-1, N-1)$.
- **State space.** The state space S consists of all non-obstacle cells: $S = \{(r, c) \mid 0 \leq r < N, 0 \leq c < N\} \setminus O$. A state is $s = (r, c)$, where r is the row index and c is the column index.
- **Action space.** $A = \{\text{Up, Down, Left, Right}\}$.
- **Transitions.** Dynamics are stochastic. Let p be the slip probability. When taking action a , the agent moves in the intended direction with probability $1-p$, and slips to one of the two orthogonal (lateral) directions with probability $p/2$ each. If a move (intended or slipped) points outside the grid boundaries or into an obstacle O , the agent simply remains in its current state.

• Reward.

$$R(s, a, s') = \begin{cases} +25, & \text{if } s' = g \\ -5, & \text{if } s' \in H \\ -1, & \text{otherwise} \end{cases} \quad (1)$$

This establishes a base step cost of -1 , an additional penalty of -4 for terminating a step in a hazard cell (H), and a $+25$ reward for reaching the goal.

Episodes terminate immediately upon reaching the goal g or after a maximum limit of $H_{\max} = 4N$ steps. This problem supports: (1) deterministic feasibility planning with explicit constraints (ideal for CSP), (2) model-based stochastic planning with full transition probabilities (perfect for MDP), and (3) model-free learning from sampled interaction (suitable for RL).

2 Methods

2.1 Approach A: CSP Replanning

We model a deterministic plan as a sequence of state variables $(X_0, X_1, \dots, X_T)$ with domains $D_t \subseteq S$, subject to the following constraints:

$$X_0 = s_{\text{current}}, X_T = g, X_t \notin O, X_t \notin H, \text{Adj}(X_t, X_{t+1}) \quad (2)$$

where s_{current} is the agent's current state, O is the set of obstacles, H is the set of hazards, and $\text{Adj}(X_t, X_{t+1})$ ensures that X_{t+1} is a valid deterministic single-step orthogonal move from X_t . At each step, the agent solves for the shortest feasible trajectory to the goal that strictly avoids all obstacles and hazards. It executes the first action of this plan and immediately replans from the new state if a stochastic slip causes a deviation.

2.2 Approach B: MDP Value Iteration

We compute the optimal policy by solving the known MDP using value iteration. We iteratively update the state values $V_k(s)$:

$$V_{k+1}(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')] \quad (3)$$

where $P(s'|s, a)$ is the exact transition probability model mapping the intended and slip dynamics, $R(s, a, s')$ is the

reward function defined earlier, and $\gamma = 0.95$ is the discount factor. The optimal policy is extracted greedily: $\pi^*(s) = \arg \max_{a \in A} Q^*(s, a)$.

2.3 Approach C: RL Q-Learning

We train a model-free tabular Q-learning agent that learns strictly from environment interaction via an ϵ -greedy exploration strategy. The action-value function is updated after observing the reward r and the next state s' :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a' \in A} Q(s', a') - Q(s, a) \right] \quad (4)$$

We use a constant learning rate of $\alpha = 0.2$, a discount factor of $\gamma = 0.95$, and a linearly decaying exploration rate ϵ that drops from 1.0 to 0.05 over the course of training.

3 Experimental Design

The experiments were conducted on three main scenarios: Easy ($N = 10, p = 0.05, o = 0.12, h = 0.08$), Medium ($N = 12, p = 0.15, o = 0.18, h = 0.10$), Hard ($N = 14, p = 0.25, o = 0.22, h = 0.12$), where o is the density of obstacles and h is the density of hazards. For each scenario and method, we evaluate 120 episodes and record:

- success rate,
- average return,
- average steps on successful episodes,
- total runtime (preparation plus evaluation).

Additional runs on $N \in \{8, 10, 12, 14, 16\}$ (80 episodes each) measure runtime growth and success stability (see Figure 1). Additionally, all random generators use fixed seeds. The repository provides a single command to regenerate all tables and figures.

4 Results

Table 1 summarizes the three main scenarios.

Table 1: Main scenario results.

Scenario	Method	Success	Return	Time(s)
Easy	CSP	1.0000	6.55	0.2562
Easy	MDP	1.0000	7.02	0.0322
Easy	RL	1.0000	6.79	0.1755
Medium	CSP	1.0000	-1.44	0.4504
Medium	MDP	1.0000	-0.01	0.0565
Medium	RL	1.0000	-0.96	0.2336
Hard	CSP	0.1583	-57.47	1.2358
Hard	CSP (relaxed)	0.9916	-31.71	1.47
Hard	MDP	0.9833	-24.48	0.3548
Hard	RL	0.0000	-56.10	0.4086

strictly avoiding hazards for CSP makes it fail more often under high slip and dense hazards. However, introducing a second CSP that relaxes the hazard constraint if no path is found, restores navigability on the hard setting (boosting success to 0.9916), though at the cost of a lower average return (-31.71) compared to MDP and a longer runtime (1.47s). The

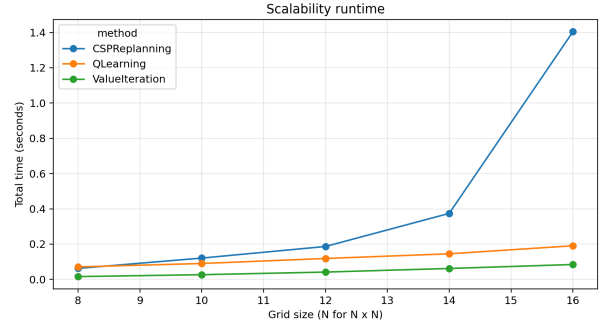


Figure 1: Scalability runtime versus grid size.

results show that MDP gives the strongest robustness under high stochasticity (hard success rate 0.9833). CSP replanning is competitive on easy and medium settings, but it fails more often under high slip and dense hazards. RL reaches strong performance on easy and medium settings, but the hard setting shows that finite tabular training can be insufficient in sparse-reward, high-uncertainty conditions.

5 Parameter Sensitivity Analysis

We performed a one-factor-at-a-time sweep (5 values per parameter) and report only the parameter variations that produced a clear behavioral change in success, return, or runtime.

Q-learning training budget (ql.episodes). On the medium scenario, increasing training from 500 to 1000 episodes changes success from 0.8167 to 1.0000, and improves return from -16.54 to -2.07. Additional training (1500–4000) mostly preserves success while slightly improving return around the same range. On the hard scenario, however, success remains 0.0000 across all tested budgets, indicating that budget alone does not solve the hardest regime. As expected, preparation time grows almost linearly with the number of episodes.

Q-learning learning rate (ql.alpha). This parameter strongly affects stability on medium difficulty. At $\alpha = 0.05$, success drops to 0.1917 with a return of -41.96, while values in $[0.1, 0.5]$ recover to 1.0000 success and much higher returns (between roughly -1.93 and -0.93). This indicates that too small an update step prevents effective policy improvement within the fixed training horizon.

Q-learning discount factor (ql.gamma). On medium difficulty, changing γ produces a meaningful return variation (from -2.28 at 0.85 to -1.10 at 0.98) while keeping success at 1.0000. This confirms that long-term credit assignment remains important even when goal reachability is already high.

Q-learning exploration decay (ql.epsilon_decay). On hard difficulty, this is the only exploration-related parameter that produced a detectable change in success: with decay 0.99, success reaches 0.0083, whereas the other tested decays stay at 0.0000. The effect is small but measurable, suggesting that exploration schedule shape is more influential than start or floor values in this setting.

Value Iteration discount factor (`vi_gamma`). This is the dominant MDP sensitivity on the hard scenario. Success rises from 0.0000 ($\gamma = 0.85$) to 0.1917 ($\gamma = 0.90$), then to 0.9917 ($\gamma = 0.95$), and reaches 1.0000 for $\gamma \in \{0.98, 0.995\}$. At the same time, preparation time increases sharply (about 0.08s at 0.85 versus about 0.96s at 0.995), highlighting a clear robustness-versus-runtime trade-off.

Value Iteration convergence threshold (`vi_theta`). Across tested values, success and return remain nearly unchanged, but runtime changes substantially: on hard, preparation time drops from about 0.289s at 10^{-7} to about 0.131s at 10^{-3} . This parameter is therefore most useful as a computational tuning knob rather than a policy-quality knob in our tested regime.

6 Qualitative Comparison

Relaxing these safety constraints restores feasibility but leads to sub-optimal paths that accumulate hazard penalties. **CSP replanning** has the strength of transparent constraints and direct control over safety rules. However, it proves brittle when uncertainty causes frequent replanning or when strict hazard avoidance removes many feasible trajectories. In contrast, **MDP value iteration** explicitly optimizes the expected long-term return under stochastic transitions, providing strong robustness. Its main drawback is the requirement for a full transition and reward model, alongside a computational cost that grows directly with the state space size. Finally, **Q-learning** offers the advantage of requiring no prior model, allowing it to adapt purely from interaction. On the downside, it is highly sensitive to the exploration schedule and training budget, and its tabular representation scales poorly as the environment grows.

When to use what. In summary, one should use CSP when safety constraints are strict and dynamics are mostly predictable. MDP is preferable when a reliable environment model is available and handling uncertainty optimally is critical. RL remains the method of choice when prior model knowledge is unavailable and online adaptation is required.

7 Technical Quality and Reproducibility

The codebase is modular (environment, methods, evaluation, runner), deterministic via fixed seeds, and fully reproducible from command line.

8 Conclusion

Across identical tasks, MDP planning is the best overall trade-off in this project, especially on the hard stochastic setting. CSP replanning is fast to implement and interpretable but less robust at higher uncertainty. Q-learning is promising but needs more sample-efficient function approximation for difficult settings.

References